10

Infrastructure Automation

Describing infrastructure as code with Terraform, and deploying packaged solutions from Cloud Marketplace

PCA study series · 7 concept slides · 7 scenario questions

Clicking and scripting versus declaring the end state

Imperative tools

Console, gcloud CLI, Cloud Shell

- You issue each step in order
- Great for exploring and one-off fixes
- Hard to repeat exactly across environments
- Changes leave little reviewable history

Infrastructure as code

Declarative configuration files

- You describe what should exist; the tool works out the steps
- Same files create dev, test and prod consistently
- Version control gives review, history and rollback
- Environments can be destroyed and rebuilt on demand

EXAM LENS 'Repeatable', 'consistent across environments', 'auditable changes' → infrastructure as code.

The Terraform building blocks

1 Provider

A plugin that talks to an API — google and google-beta for Google Cloud. Terraform also supports other clouds and SaaS.

2 Resource

One piece of infrastructure, such as a network or a VM, with its arguments.

3 Variable

An input that lets one configuration serve several environments.

4 Output

A value exported after apply, such as an IP address, for people or other configurations.

5 Module

A reusable bundle of resources — your own or from a registry.

6 State

Terraform's record of what it manages, used to compute the difference between code and reality.

The Terraform workflow

1

Write

Declare resources in .tf files written in HCL and commit them to a repository.

2

init

Downloads providers and modules and configures the state backend.

3

plan

Shows exactly what will be created, changed or destroyed — review before touching anything.

4

apply

Makes the changes and updates state. Dependencies are ordered automatically.

5

destroy

Removes everything the configuration manages — ideal for temporary environments.

EXAM LENS 'Preview the impact before changing production' → terraform plan.

Example: a network that accepts web traffic

```
provider "google" {
  project = var.project_id
  region  = "europe-west1"
}

resource "google_compute_network" "web_vpc" {
  name                    = "web-vpc"
  auto_create_subnetworks = true
}

resource "google_compute_firewall" "allow_web" {
  name    = "web-vpc-allow-http"
  network = google_compute_network.web_vpc.self_link

  allow {
    protocol = "tcp"
    ports    = ["80", "443"]
  }
  source_ranges = ["0.0.0.0/0"]
  target_tags   = ["web"]
}
```

Implicit dependency
The firewall refers to the network's self_link, so Terraform creates the network first.

Targeted rule
target_tags limits the rule to VMs tagged web.

Parameterised
var.project_id lets the same file deploy to any project.

Running Terraform as a team

1 Remote state

Store state in a versioned Cloud Storage bucket. The backend locks state so two runs don't collide.

2 Least-privilege identity

Run Terraform as a dedicated service account with only the roles it needs; avoid downloaded keys.

3 Pipelines

Trigger plan on pull request and apply on merge with Cloud Build or another CI system.

4 Managed execution

Infrastructure Manager runs Terraform for you on Google Cloud, keeping state and logs.

EXAM LENS Manual console changes cause drift; the next plan shows them and apply restores the declared state.

Cloud Marketplace: production-ready solutions in a few clicks

When the need is a known product, deploying a vetted package beats building it from scratch.

What's there	VM images, Kubernetes apps, SaaS and datasets from Google and partners.
How it deploys	Packaged as Terraform, launched in your project with sensible defaults you can adjust.
Billing	Licences can be pay-as-you-go on your Google Cloud bill, or bring-your-own.
Updates	Vendors publish new versions; you control when to redeploy.

Decision cheat sheet: automation

If the scenario says…	Reach for
Identical dev, test and prod environments	Terraform modules with per-environment variables
See changes before applying	terraform plan, reviewed in a pull request
Several engineers run Terraform	Remote state in Cloud Storage with locking
Tear down a temporary environment completely	terraform destroy
Managed Terraform runs on Google Cloud	Infrastructure Manager
Deploy a known third-party product quickly with its licence billed by Google	Cloud Marketplace
Quick one-off investigation	Console, gcloud or Cloud Shell



7 PCA-style scenarios

Read every requirement word, eliminate options that break one, then pick the most managed answer that satisfies them all. Each question is followed by its answer and the reasoning.

Question 1 of 7

Environments

A platform team must create dev, staging and production environments that are identical except for machine sizes and project IDs. Every change must be reviewed before it reaches production.

A Build production in the console and write a runbook for the others

B Write Terraform modules with variables per environment, store them in Git and apply through a reviewed pipeline

C Copy production VMs into the other projects with machine images

D Use gcloud scripts stored on an engineer's laptop

Answer 1: B

B Write Terraform modules with variables per environment, store them in Git and apply through a reviewed pipeline

Why it fits

One set of modules parameterised by variables guarantees consistency, and Git-based review gives an auditable approval path.

Why the others don't

A — Manual steps drift between environments.

C — Copies VMs only, not networks, IAM and other resources.

D — Imperative, unreviewed and not shared.

EXAM LENS 'Identical environments' + 'reviewed' → IaC modules in version control.

Question 2 of 7

State

Two engineers ran terraform apply at the same time against the same project and corrupted the configuration. How should the team prevent this?

A. Keep state files in each engineer's home directory

B. Configure a remote backend in a versioned Cloud Storage bucket, which provides state locking

C. Commit the state file to Git after each run

D. Use the console for changes instead

Answer 2: B

B Configure a remote backend in a versioned Cloud Storage bucket, which provides state locking

Why it fits

A shared remote backend gives one source of truth, locks state during runs and keeps prior versions for recovery.

Why the others don't

A — Each engineer sees a different reality.

C — Git doesn't lock state and state can contain secrets.

D — Abandons automation instead of fixing it.

EXAM LENS Team + Terraform → remote state with locking.

Question 3 of 7

Preview

Before changing a production load balancer's configuration, the change advisory board wants to see exactly which resources will be modified or replaced.

A terraform init

B terraform plan

C terraform apply -auto-approve

D terraform destroy

Answer 3: B

B terraform plan

Why it fits

plan compares the configuration with current state and lists every create, update and replace without making changes.

Why the others don't

A — Only prepares providers and backend.

C — Makes changes immediately without review.

D — Deletes the infrastructure.

EXAM LENS 'Preview' / 'what will change' → plan.

Question 4 of 7

Drift

An operator manually opened port 22 on a firewall rule that Terraform manages. What happens on the next terraform plan and apply?

A. Terraform ignores changes made outside it

B. plan reports the drift and apply returns the rule to the state defined in code

C. Terraform deletes the whole VPC

D. apply fails permanently until the state file is deleted

Answer 4: B

B plan reports the drift and apply returns the rule to the state defined in code

Why it fits

Terraform refreshes real infrastructure, detects the difference from configuration and plans to restore the declared settings.

Why the others don't

A — Managed resources are always compared.

C — Only the drifted attribute is reconciled.

D — No failure; drift is normal to resolve.

EXAM LENS Code is the source of truth; manual changes are reverted.

Question 5 of 7

Marketplace

A team needs a commercial next-generation firewall appliance running in its VPC this week, with licence charges appearing on the Google Cloud invoice.

A Build a custom image from the vendor's ISO

B Deploy the vendor's solution from Cloud Marketplace

C Use Cloud NAT

D Write a Terraform module from scratch

Answer 5: B

B Deploy the vendor's solution from Cloud Marketplace

Why it fits

Marketplace offers vendor-packaged deployments with pay-as-you-go licensing billed through Google Cloud, deployable in minutes.

Why the others don't

A — Slow and leaves licensing to sort out separately.

C — Not a firewall appliance.

D — Reinvents what the vendor already packages.

EXAM LENS 'Third-party product' + 'quickly' + 'billed by Google' → Cloud Marketplace.

Question 6 of 7

Clean-up

A training environment of 40 resources is created by Terraform every Monday and must be removed completely every Friday so no costs remain.

A Delete the project manually each Friday

B Run terraform destroy from the same configuration on Friday

C Stop all VMs on Friday

D Lower the quota to zero

Answer 6: B

B Run terraform destroy from the same configuration on Friday

Why it fits

destroy removes every resource the configuration manages in dependency order, leaving nothing to bill.

Why the others don't

A — Works, but loses project setup and isn't repeatable.

C — Stopped VMs still incur disk and IP charges.

D — Blocks new resources but doesn't delete existing ones.

EXAM LENS 'Remove everything Terraform created' → destroy.

Question 7 of 7

Managed runs

Security wants Terraform deployments to run inside Google Cloud with state and logs managed for them, without operating their own runner VMs.

A Cloud Shell

B Infrastructure Manager

C Cloud Marketplace

D Compute Engine with cron

Answer 7: B

B Infrastructure Manager

Why it fits

Infrastructure Manager is Google's managed service for running Terraform, storing state and recording deployment history.

Why the others don't

A — Interactive and tied to a user session.

C — For packaged products, not your own configurations.

D — Exactly the runner they want to avoid managing.

EXAM LENS 'Managed Terraform on Google Cloud' → Infrastructure Manager.